1. University Scholarship Allocation

1. Problem Statement

Design a system that evaluates scholarship applicants using aggregate marks, subject-wise minimum scores, attendance, and annual family income. It identifies eligible students, ranks them by merit, and resolves ties using predefined rules.

2. Objectives

1. Check aggregate and minimum subject marks.
2. Verify attendance and family-income criteria.
3. Classify applicants as eligible or ineligible.
4. Rank eligible students and apply tie-breaking rules.
5. Print a merit list and scholarship allocation summary.

3. Assumptions

• Aggregate minimum: 75%.

• Each subject minimum: 60%.

• Attendance minimum: 75%.

• Annual family income maximum: ■250,000.

• All conditions must be met; ranking is aggregate descending, attendance descending, lowest subject mark descending, then student ID ascending.

• Scholarships go to the top five eligible students.

4. Algorithm

1. Read applicant count and each student's ID, name, subject marks, attendance, and annual income.
2. Calculate average marks and lowest subject mark.
3. Check every eligibility condition.
4. Separate eligible and ineligible students.
5. Sort eligible applicants by the specified ranking and tie-breaking rules.
6. Print the merit list and allocation count.

5. Pictorial Flowchart

START ↓ Input student details ↓ Calculate aggregate ↓ Check marks, attendance and income ↓ Eligible? — Yes: add to merit list; No: mark ineligible ↓ Sort eligible students and resolve ties ↓ Display top five and summary ↓ END

6. Python Source Code

```
MIN_AGGREGATE = 75
MIN_SUBJECT = 60
MIN_ATTENDANCE = 75
MAX_INCOME = 250000
SCHOLARSHIP_LIMIT = 5

def evaluate(s):
    marks = s["marks"]
    s["aggregate"] = sum(marks) / len(marks)
    s["lowest"] = min(marks)
    s["eligible"] = (
        s["aggregate"] >= MIN_AGGREGATE
        and all(m >= MIN_SUBJECT for m in
        marks) and s["attendance"] >=
        MIN_ATTENDANCE and s["income"] <=
        MAX_INCOME
    )
    return s

def main():
    students = []
    n = int(input("Number of applicants:
    ")) if n <= 0:
        print("Applicant count must be
        positive.") return
```

```
    for i in range(n):
        print("\nApplicant", i + 1)
        sid = input("Student ID:
        ").strip() name = input("Name:
        ").strip()
        count = int(input("Number of subjects:
        )) if count <= 0:
            print("Subject count must be positive.")
            return
        marks = [float(input(f"Subject {j+1} marks (0-100):
                 )) for j in range(count)]
        attendance = float(input("Attendance (0-100):
        )) income = float(input("Annual family
        income: )) if (any(m < 0 or m > 100 for m in
        marks)
                or not 0 <= attendance <= 100 or income < 0):
            print("Invalid input.")
            return
        students.append(evaluate({
            "id": sid, "name": name, "marks":
            marks, "attendance": attendance,
            "income": income
        }))

    eligible = [s for s in students if s["eligible"]]
    eligible.sort(key=lambda s: (
        -s["aggregate"], -s["attendance"], -s["lowest"], s["id"].lower()
    ))
    print("\nMERIT LIST")
    print(f"{'Rank':<6}{'ID':<12}{'Name':<18}{'Aggregate':>10
    }") for rank, s in
    enumerate(eligible[:SCHOLARSHIP_LIMIT], 1):
        print(f"{rank:<6}{s['id']:<12}{s['name']:<18}{s['aggregate']:>10.2f
    }") print("\nIneligible applicants:")
    for s in students:
        if not s["eligible"]:
            print(s["id"], "-",
            s["name"])
    print("Total:", len(students), "| Eligible:", len(eligible),
          "| Scholarships:", min(len(eligible), SCHOLARSHIP_LIMIT))

if __name__ == "__main__":
    main()
```

7. Test Cases

Case	Input / condition	Expected result	Purpose
TC1	85	75	90
TC2	70	65	90
TC3	80	55	90
TC4	80	65	70
TC5	80	65	90
TC6	75	60	75

8. Simple Sample Output

```
Number of applicants: 2
Applicant 1: S101, Arun; marks 85, 90, 80; attendance 90; income 200000


Rank  ID          Name              Aggregate
1     S101        Arun                   85.00
2     S102        Bina                   80.00
Applicant 2: S102, Bina; marks 75, 80, 85; attendance 85; income

180000 MERIT LIST



Total: 2 | Eligible: 2 | Scholarships: 2
```

9. Conclusion

The system automates eligibility checks and merit ranking, making allocation more consistent and transparent. Real deployments should use approved institutional rules and secure student-data handling.

1. University Scholarship Allocation

10. Output Screenshot

Screenshot of the program run in a terminal using the sample input from Section 8.

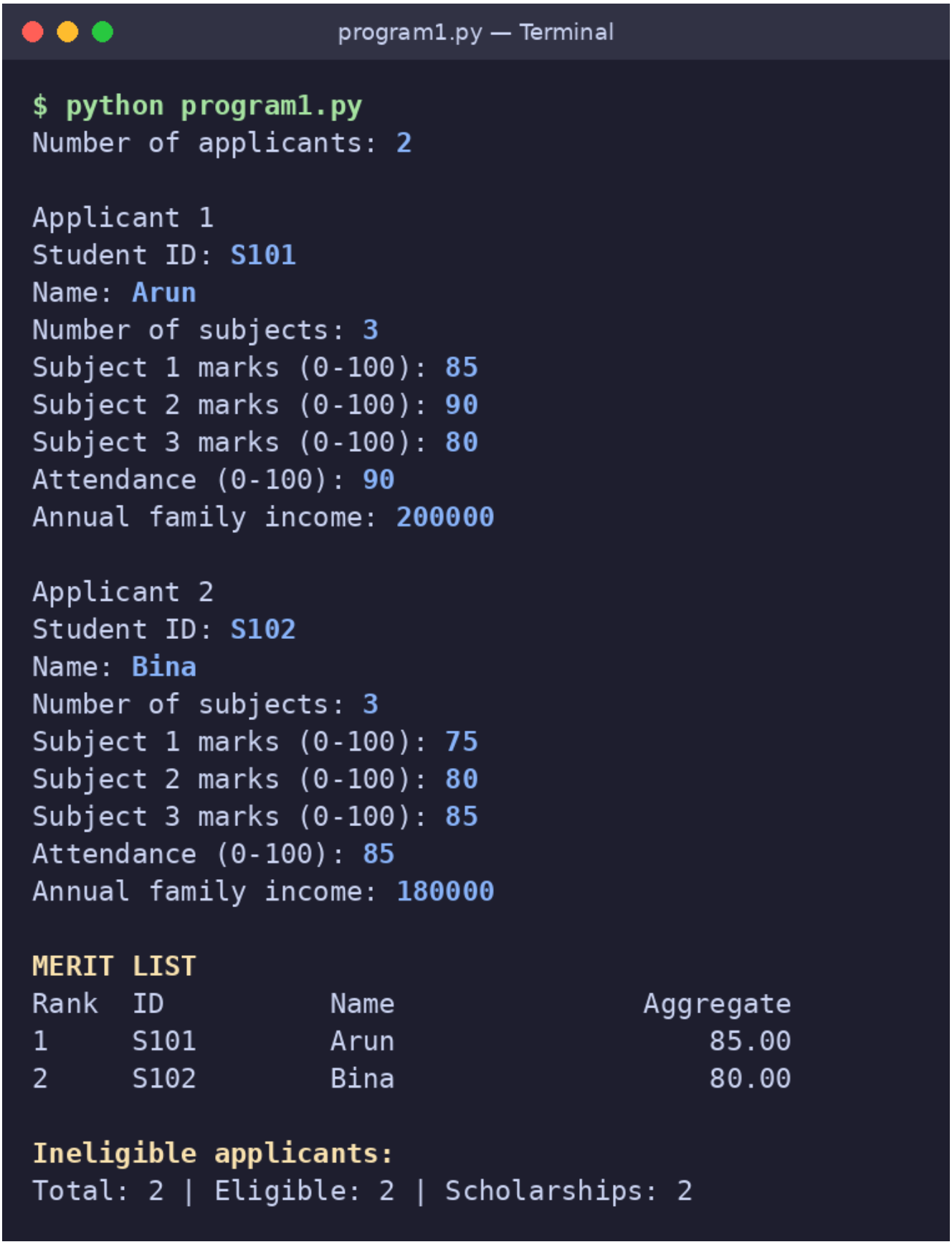


Figure 1: Console output of Program 1 (University Scholarship Allocation).

2. Progressive Electricity Billing

1. Problem Statement

Calculate monthly household electricity bills using progressive consumption slabs, fixed charges, surcharges, and rebates. Compare bills across months and flag unusual increases.

2. Objectives

1. Calculate energy charges slab by slab.
2. Add fixed charges and surcharges, then apply eligible rebates.
3. Compare several monthly bills.
4. Flag a month whose bill rises above a defined percentage.

3. Assumptions

• Illustrative rates only: first 100 units ■1.50/unit; next 100 ■2.50/unit; above 200 ■4/unit.

• Fixed charge ■75/month.

• Surcharge is 5% of energy charges.

• Rebate is 3% of energy charges when consumption is at most 100 units.

• An increase above 30% from the preceding month is unusual.

4. Algorithm

1. Input consumption for each month.
2. Calculate progressive slab charges.
3. Add fixed charge and surcharge; subtract applicable rebate.
4. Store and display each bill.
5. Compare each month with the previous month and flag increases above 30%.

5. Pictorial Flowchart

START ↓ Input monthly units ↓ Calculate charges by slab ↓ Add fixed charge and surcharge ↓ Apply rebate if eligible ↓

Compare with previous month ↓ Flag unusual increase and print bills ↓ END

6. Python Source Code

```
def
    energy_charge(units):
    if units < 0:
        raise ValueError("Units cannot be negative")
    charge = min(units, 100) * 1.50
    if units > 100:
        charge += min(units - 100, 100) *
    2.50 if units > 200:
        charge += (units - 200) * 4.00
    return charge

def bill(units):
    energy =
    energy_charge(units) fixed
    = 75.0
    surcharge = 0.05 * energy
    rebate = 0.03 * energy if units <= 100 else 0.0
    return round(energy + fixed + surcharge - rebate,
    2)

def main():
    n = int(input("Number of months:
    ")) if n <= 0:
        print("Enter at least one month.")
        return
    bills = []
    for i in range(n):
        units = float(input(f"Month {i+1} consumption (units):
        ")) if units < 0:
            print("Consumption cannot be
            negative.") return
        bills.append((units,
    bill(units))) print("\nMonth Units
                          Bill")
```

```
for i, (units, amount) in enumerate(bills,
    1): note = ""
    if i > 1:
```

```
            previous = bills[i-2][1]
            if previous > 0 and (amount-previous)/previous >
                0.30: note = " UNUSUAL INCREASE"
        print(f"{i:<6} {units:<7.1f} ■{amount:.2f}{note}")

if    name   == "   main
      ": main()
```

7. Test Cases

Case	Input / condition	Expected result	Purpose
TC1	80	■ 188.28	Rebate applies
TC2	100	■ 216.45	Slab boundary
TC3	150	■ 338.75	Two slabs
TC4	250	■ 610.00	Three slabs
TC5	-5	—	Reject negative usage
TC6	Month rises 100 to 250 units	Compare totals	Flag if bill rises >30%

8. Simple Sample Output

```
Number of months: 3
Month 1 consumption (units): 80
Month 2 consumption (units): 100
Month 3 consumption (units): 250

Month  Units   Bill
1      80.0    ■ 188.28
2      100.0   ■ 216.45 UNUSUAL INCREASE
3      250.0   ■ 610.00 UNUSUAL INCREASE
```

9. Conclusion

The program models progressive tariffs and helps households monitor spending. Actual electricity rates and rebate rules must be replaced with the local utility's current tariff.

2. Progressive Electricity Billing

10. Output Screenshot

Screenshot of the program run in a terminal using the sample input from Section 8.

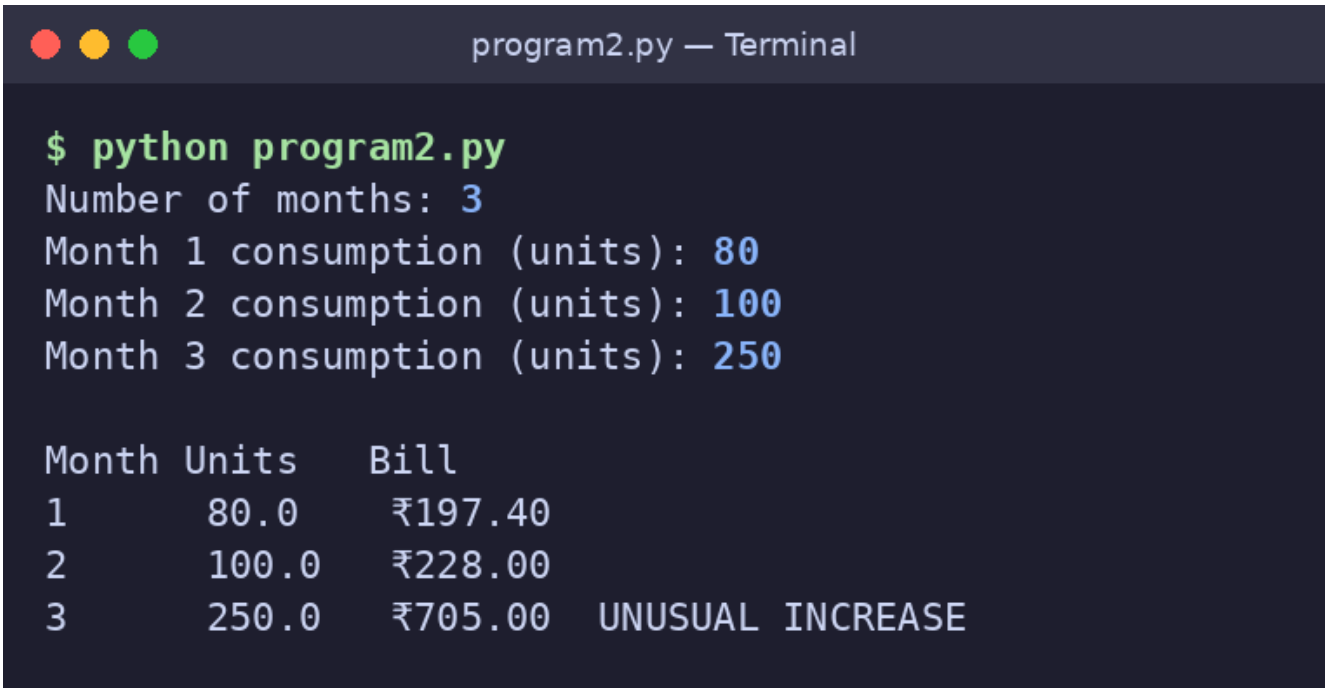


Figure 2: Console output of Program 2 (Progressive Electricity Billing).

3. Hospital Billing and Patient Classification

1. Problem Statement

Process consultation, diagnostic, and treatment charges; apply conditional concessions; validate billing constraints; and classify patients according to a stated payment policy.

2. Objectives

1. Compute itemized hospital charges.
2. Apply concessions based on patient category.
3. Validate quantities and charges.
4. Classify payment status and display balance due.

3. Assumptions

• Consultation, diagnostics, and treatment charges are non-negative.

• Senior citizens receive a 10% concession on consultation charges.

• Low-income patients receive a 15% concession on treatment charges.

• Tax is 5% after concessions.

• Payment classification: paid in full = Settled; partial payment = Partially Paid; zero payment = Unpaid.

4. Algorithm

1. Input patient category, service charges, and amount paid.
2. Reject negative charges or payment.
3. Calculate applicable concessions.
4. Calculate taxable subtotal and tax.
5. Compute total and outstanding balance.
6. Classify payment status and print invoice.

5. Pictorial Flowchart

START ↓ Input patient and charges ↓ Validate non-negative amounts ↓ Apply eligible concessions ↓ Calculate subtotal, tax and total ↓ Compare payment with total ↓ Print bill and classification ↓ END

6. Python Source Code

```
def hospital_bill(consultation, diagnostics, treatment,
                  senior=False, low_income=False,
    paid=0): values = [consultation, diagnostics,
    treatment, paid] if any(v < 0 for v in values):
        raise ValueError("Amounts cannot be negative")
    c_discount = consultation * 0.10 if senior else 0
    t_discount = treatment * 0.15 if low_income else 0
    subtotal = consultation + diagnostics + treatment - c_discount -
    t_discount tax = subtotal * 0.05
    total = round(subtotal + tax,
    2) if paid > total:
        raise ValueError("Payment cannot exceed the bill
    total") balance = round(total - paid, 2)
    status = "Settled" if balance == 0 else (
        "Partially Paid" if paid > 0 else
        "Unpaid")
    return total, balance, status, c_discount, t_discount, tax

def main():
    consultation = float(input("Consultation charge: ■"))
    diagnostics = float(input("Diagnostic charges: ■"))
    treatment = float(input("Treatment charges: ■"))
    senior = input("Senior citizen? (y/n): ").lower() == "y"
    low_income = input("Low-income concession? (y/n): ").lower() ==
    "y" paid = float(input("Amount paid: ■"))
    try:
        total, balance, status, cd, td, tax = hospital_bill(
            consultation, diagnostics, treatment, senior, low_income, paid)
    except ValueError as e:
        print("Invalid bill:",
        e) return
```

```
    print("\nHOSPITAL BILL")
    print(f"Consultation concession:
    ■{cd:.2f}") print(f"Treatment concession:
    ■{td:.2f}") print(f"Tax: ■{tax:.2f}")
    print(f"Total: ■{total:.2f}")
    print(f"Balance: ■{balance:.2f}")
    print(f"Classification: {status}")

if   name   == "   main
      ": main()
```

7. Test Cases

Case	Input / condition	Expected result	Purpose
TC1	■500+■800+■2,000; no concessions; paid ■0	▪ 3,465.00	Unpaid
TC2	Senior; consultation ■1,000	▪ 100 concession before tax	Valid concession
TC3	Low income; treatment ■2,000	▪ 300 concession before tax	Valid concession
TC4	Negative diagnostic charge	—	Reject input
TC5	Payment equals total	Balance ■0	Settled
TC6	Payment less than total	Positive balance	Partially Paid

8. Simple Sample Output

```
Consultation charge:
■500 Diagnostic
charges: ■800 Treatment
charges: ■2000 Senior
citizen? (y/n): n
Low-income concession? (y/n): n
Amount paid: ■1000

HOSPITAL BILL
Consultation concession:
■0.00 Treatment concession:
■0.00 Tax: ■165.00
Total: ■3465.00
Balance: ■2465.00
Classification: Partially Paid
```

9. Conclusion

The system calculates transparent hospital bills, applies configured concessions, and classifies payment status. Hospitals should configure it to match approved billing and assistance policies.

3. Hospital Billing and Patient Classification

10. Output Screenshot

Screenshot of the program run in a terminal using the sample input from Section 8.

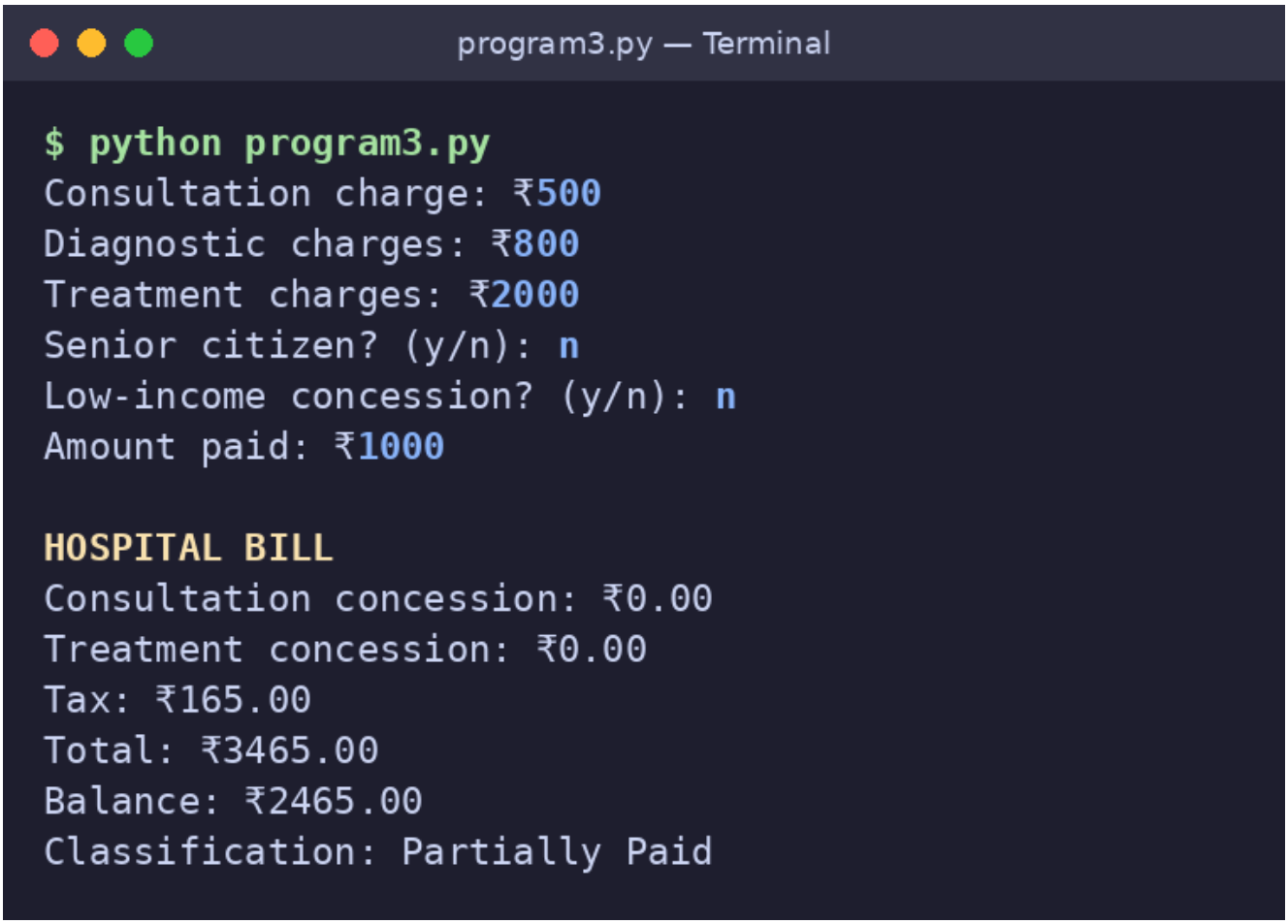


Figure 3: Console output of Program 3 (Hospital Billing and Patient Classification).

4. Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions; and generate a salary statement showing gross pay, deductions, and net pay.

2. Objectives

1. Compute earnings and gross salary.
2. Apply progressive income-tax-style deductions.
3. Include fixed deductions.
4. Generate a formatted salary statement.

3. Assumptions

• All figures are monthly and in INR.

• HRA is entered as an amount; overtime pay is hours × overtime rate.

• Illustrative progressive deduction on gross pay: 0% up to ■25,000; 5% on next ■25,000; 10% on amount above ■50,000.

• Fixed deduction is 2% of basic pay.

• Rates are educational examples, not actual tax advice.

4. Algorithm

1. Read basic pay, allowances, overtime hours/rate, incentives.
2. Validate inputs.
3. Calculate overtime and gross pay.
4. Calculate progressive deduction by bands.
5. Calculate fixed deduction and net salary.
6. Print the salary statement.

5. Pictorial Flowchart

START ↓ Input salary components ↓ Validate inputs ↓ Calculate overtime and gross pay ↓ Calculate progressive and fixed deductions ↓ Calculate net pay ↓ Print salary statement ↓ END

6. Python Source Code

```
def progressive_deduction(gross):
    first = min(gross, 25000) *
    0.00
    second = min(max(gross - 25000, 0), 25000) * 0.05
    third = max(gross - 50000, 0) * 0.10
    return first + second + third

def main():
    basic = float(input("Basic pay: ■"))
    allowances = float(input("Total allowances: ■"))
    hours = float(input("Overtime hours: "))
    rate = float(input("Overtime rate per hour: ■"))
    incentive = float(input("Incentives: ■"))
    if min(basic, allowances, hours, rate, incentive) < 0:
        print("All values must be non-negative.")
        return
    overtime = hours * rate
    gross = basic + allowances + overtime + incentive
    progressive = progressive_deduction(gross)
    fixed = basic * 0.02
    deductions = progressive +
    fixed net = gross - deductions
    print("\nSALARY STATEMENT")
    print(f"Basic pay:          ■{basic:.2f}")
    print(f"Allowances:         ■{allowances:.2f}")
    print(f"Overtime:           ■{overtime:.2f}")
    print(f"Incentives:         ■{incentive:.2f}")
    print(f"Gross salary:       ■{gross:.2f}")
    print(f"Progressive deduction:
    ■{progressive:.2f}") print(f"Fixed deduction:
    ■{fixed:.2f}") print(f"Total deductions:
    ■{deductions:.2f}")
```

```
    print(f"Net salary:         ■{net:.2f}")

if    name    == "    main
      ": main()
```

7. Test Cases

Case	Input / condition	Expected result	Purpose
TC1	Basic 20,000; allowances 5,000; OT 0; incentive 0	Gross ■25,000	Progressive deduction ■0
TC2	Gross ■40,000; basic ■30,000	Progressive ■750; fixed ■600	Net ■38,650
TC3	Gross ■60,000; basic ■40,000	Progressive ■2,250; fixed ■ 800	Net ■56,950
TC4	Negative basic pay	—	Reject input
TC5	Overtime 5 h × ■200	Overtime ■1,000	Included in gross
TC6	Incentive ■3,000	Gross increases by ■3,000	Recalculate deductions

8. Simple Sample Output

```
Basic pay: ■30000
Total allowances:
■10000 Overtime hours:
5
Overtime rate per hour: ■200
Incentives: ■3000

SALARY STATEMENT
Basic pay:          ■30000.00
Allowances:         ■10000.00
Overtime:           ■1000.00
Incentives:         ■3000.00
Gross salary:
■44000.00 Progressive
deduction: ■950.00 Fixed
deduction:          ■600.00
Total deductions:   ■1550.00
Net salary:
■42450.00
```

9. Conclusion

The payroll system summarizes earnings and deductions consistently. Real payroll must use current statutory tax rules, benefits, and employer policies.

4. Employee Payroll and Deduction Analysis

10. Output Screenshot

Screenshot of the program run in a terminal using the sample input from Section 8.

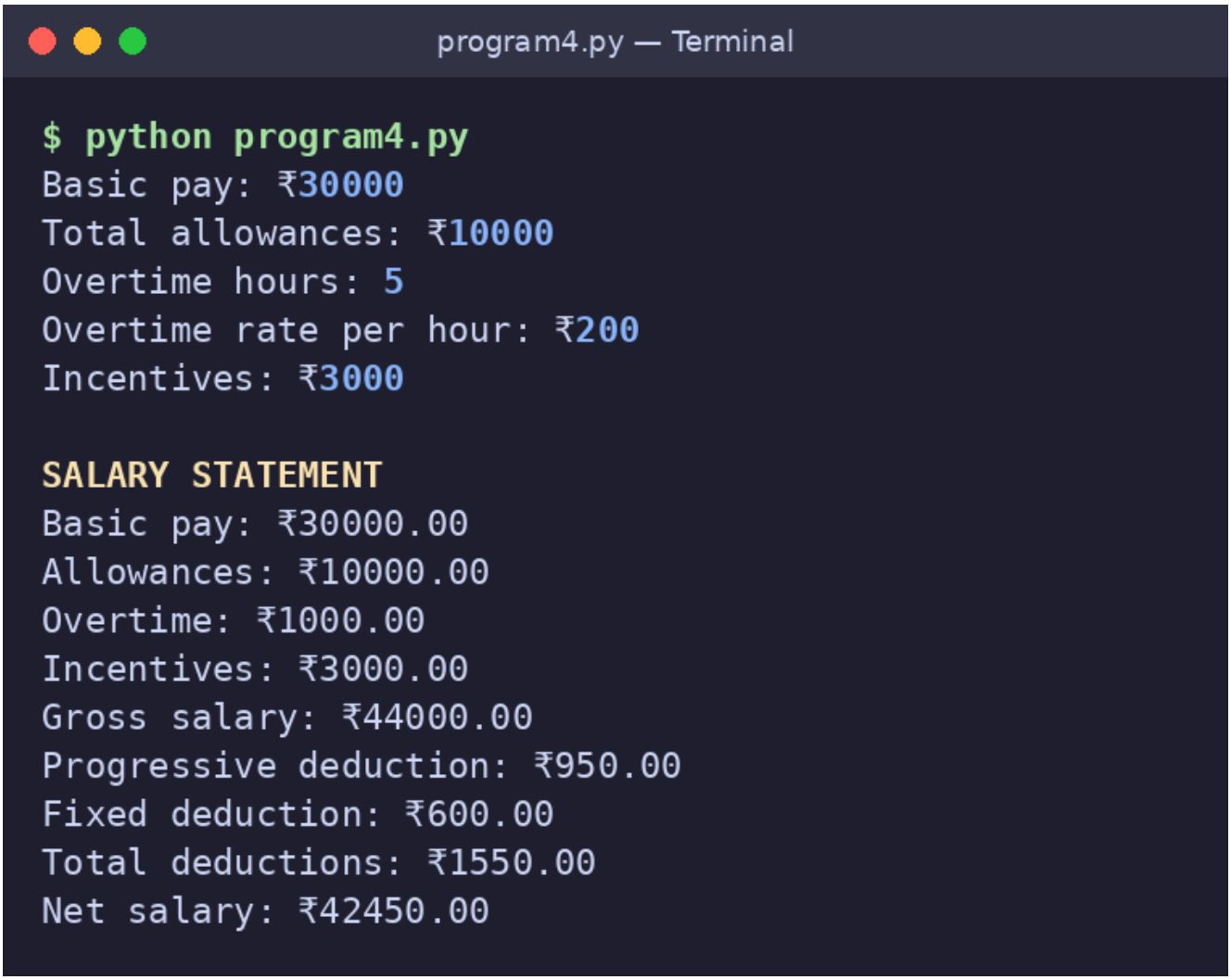


Figure 4: Console output of Program 4 (Employee Payroll and Deduction Analysis).

5. Multi-Plan Telecommunications Cost Optimization

1. Problem Statement

Compare mobile plans against a user's data, call, validity, and budget requirements. Calculate excess usage charges and identify the cheapest plan satisfying all constraints.

2. Objectives

1. Store and compare plan allowances and prices.

2. Estimate extra charges for excess data and calls.

3. Exclude plans that fail validity or budget limits.

4. Recommend the lowest-cost eligible plan.

3. Assumptions

• Each plan has a base price, included data in GB, included call minutes, validity days, data overage rate per GB, and call overage rate per minute.

• Usage beyond included allowances is billed proportionally.

• The customer specifies minimum validity and maximum budget.

• Plan values are illustrative.

4. Algorithm

1. Input customer usage, required validity, and budget.

2. Define available plans.

3. For each plan, compute excess data/calls and total cost.

4. Check budget and validity constraints.

5. Sort eligible plans by total cost.

6. Display all comparisons and the recommended plan.

5. Pictorial Flowchart

START ↓ Input usage, validity and budget ↓ Load plan definitions ↓ Calculate overage charges and total cost ↓ Check budget and validity ↓ Any eligible plans? ↓ Display cheapest eligible plan or no match ↓ END

6. Python Source Code

```
PLANS = [
    {"name": "Basic", "price": 199, "data": 2, "calls": 300,
     "days": 28, "data_rate": 50, "call_rate": 0.50},
    {"name": "Smart", "price": 299, "data": 5, "calls": 600,
     "days": 28, "data_rate": 40, "call_rate": 0.40},
    {"name": "Max", "price": 449, "data": 10, "calls": 1000,
     "days": 56, "data_rate": 30, "call_rate": 0.30},
]

def main():
    data_used = float(input("Expected data usage (GB):
    ")) call_minutes = float(input("Expected call
    minutes: ")) min_days = int(input("Minimum validity
    days: ")) budget = float(input("Maximum budget:
    ■"))
    if min(data_used, call_minutes, min_days, budget) < 0:
        print("Inputs cannot be negative.")
        return
    results = []
    for p in
    PLANS:
        extra_data = max(0, data_used - p["data"])
        extra_calls = max(0, call_minutes -
        p["calls"]) total = (p["price"] + extra_data *
        p["data_rate"]
                + extra_calls * p["call_rate"])
        ok = p["days"] >= min_days and total <= budget
        results.append((total, p, extra_data, extra_calls,
        ok))
    print("\nPLAN COMPARISON")
    for total, p, ed, ec, ok in results:
        print(f'{p["name"]}: total
```

```
    ■{total:.2f}; '
          f'extra data {ed:.2f} GB; extra calls {ec:.0f} min; '
          f'eligible={ok}')
choices = [r for r in results if r[4]]
```

```
    if choices:
        best = min(choices, key=lambda r: (r[0], r[1]["days"], r[1]["name"]))
        print(f"Recommended: {best[1]['name']} at
    ■{best[0]:.2f}") else:
        print("No plan satisfies both budget and validity.")

if    name    == "   main
      ": main()
```

7. Test Cases

Case	Input / condition	Expected result	Purpose
TC1	1 GB, 200 min, 28 days, ■250	Basic ■199	Basic recommended
TC2	4 GB, 500 min, 28 days, ■350	Smart ■299	Smart recommended
TC3	8 GB, 800 min, 50 days, ■500	Max ■449 + overages if any	Check validity and budget
TC4	Budget ■100	No plan under budget	No eligible plan
TC5	Negative usage	—	Reject input
TC6	Minimum validity 60 days	No listed plan qualifies	No eligible plan

8. Simple Sample Output

```
Expected data usage (GB): 4
Expected call minutes: 500
Minimum validity days: 28
Maximum budget: ■350

PLAN COMPARISON
Basic: total ■349.00; extra data 2.00 GB; extra calls 200 min;
eligible=True Smart: total ■299.00; extra data 0.00 GB; extra calls 0 min;
eligible=True Max: total ■449.00; extra data 0.00 GB; extra calls 0 min;
eligible=False Recommended: Smart at ■299.00
```

9. Conclusion

The optimizer compares expected usage with plan allowances and highlights the lowest-cost plan that meets constraints. Real carrier terms, fair-use policies, and taxes should be added for practical use.

5. Multi-Plan Telecommunications Cost Optimization

10. Output Screenshot

Screenshot of the program run in a terminal using the sample input from Section 8.

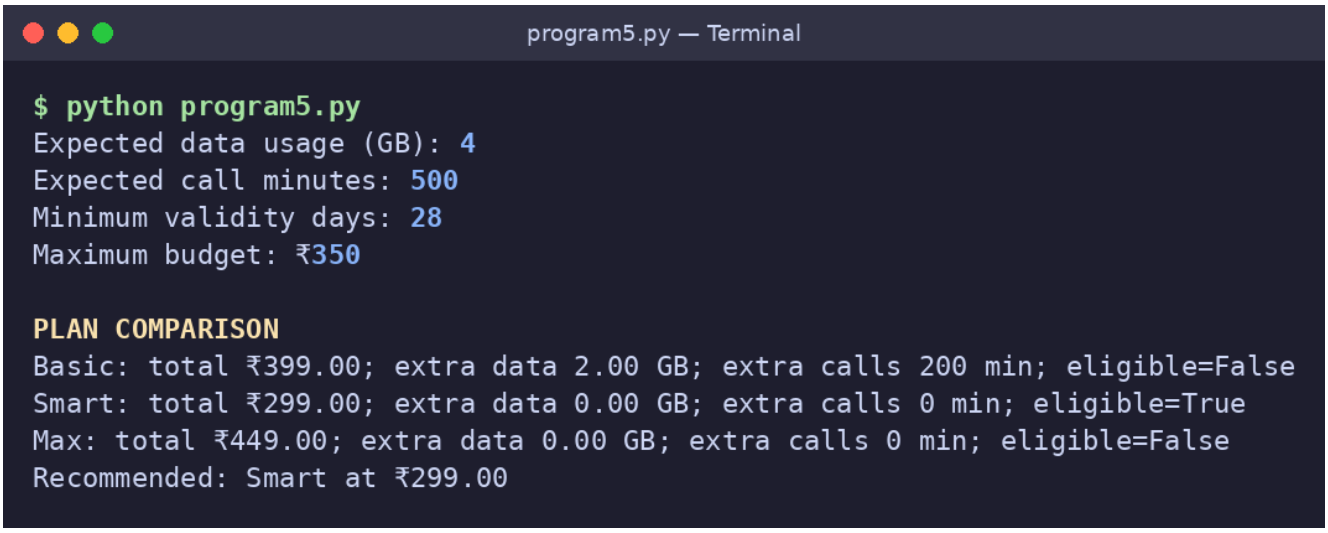


Figure 5: Console output of Program 5 (Multi-Plan Telecommunications Cost Optimization).

6. Railway Fare and Reservation Calculator

1. Problem Statement

Calculate fares for multiple passengers using distance slabs, travel classes, age concessions, reservation fees, and cancellation penalties.

2. Objectives

1. Calculate class-based distance fares.
2. Apply eligible age concessions.
3. Add reservation fees.
4. Calculate cancellation refunds and penalties.
5. Display passenger-wise and total fares.

3. Assumptions

• Illustrative rates per km: Sleeper ■0.50, AC3 ■1.20, AC2 ■1.80.

• Reservation fee: Sleeper ■20, AC3 ■40, AC2 ■50 per passenger.

• Children under 5 travel free; age 60+ receive a 20% fare concession; concession applies to distance fare only.

• Cancellation penalty is 25% of the pre-fee distance fare; reservation fee is non-refundable.

• Actual railway rules vary and are not represented.

4. Algorithm

1. Input passenger count, distance, class, age, and cancellation status.
2. Validate distance, age, and class.
3. Calculate distance fare and age concession.
4. Add reservation fee.
5. If cancelled, calculate penalty and refund.
6. Display each passenger's fare or refund.

5. Pictorial Flowchart

START ↓ Input trip and passenger details ↓ Validate entries ↓ Calculate class fare and concession ↓ Add reservation fee ↓

Cancelled? — calculate penalty/refund ↓ Display passenger totals ↓ END

6. Python Source Code

```
RATES = {"sleeper": 0.50, "ac3": 1.20, "ac2": 1.80}
FEES = {"sleeper": 20, "ac3": 40, "ac2": 50}

def calculate_fare(distance, age, travel_class,
    cancelled=False): travel_class = travel_class.lower()
    if distance < 0 or age < 0 or travel_class not in RATES:
        raise ValueError("Invalid distance, age, or class")
    if age < 5:
        base = 0.0
    else:
        base = distance * RATES[travel_class]
    concession = base * 0.20 if age >= 60 else
    0.0 discounted = base - concession
    fee = FEES[travel_class] if age >= 5 else
    0.0 total = discounted + fee
    if cancelled:
        penalty = discounted * 0.25
        refund = max(0.0, total - penalty -
        fee) return total, penalty, refund
    return total, 0.0, 0.0

def main():
    n = int(input("Number of passengers: "))
    distance = float(input("Distance in km:
    ")) if n <= 0 or distance < 0:
        print("Invalid passenger count or distance.")
        return
    grand_total = 0
```

```
    for i in range(n):
        print(f"Passenger
        {i+1}") age =
        int(input("Age: "))
        cls = input("Class (sleeper/ac3/ac2): ").lower()
        cancelled = input("Cancelled? (y/n): ").lower() == "y"
        try:
            total, penalty, refund = calculate_fare(distance, age, cls, cancelled)
        except ValueError as e:
            print(e
            )
            return
        if cancelled:
            print(f"Original fare ■{total:.2f}; penalty ■{penalty:.2f}; "
                  f"refund ■{refund:.2f}")
        else:
            print(f"Fare due: ■{total:.2f}")
            grand_total += total
    print(f"Total fare for active bookings: ■{grand_total:.2f}")

if    name   == "   main
      ": main()
```

7. Test Cases

Case	Input / condition	Expected result	Purpose
TC1	100 km, age 30, sleeper	▪ 70	▪ 50 fare + ■20 fee
TC2	100 km, age 65, AC3	▪ 136	20% concession on ■120 + ■40 fee
TC3	100 km, age 4, AC2	■0	Under-five assumption
TC4	Negative distance	—	Reject input
TC5	Cancelled sleeper, age 30, 100 km	Penalty ■12.50; refund ■37.50	Fee non-refundable
TC6	Unknown class	—	Reject input

8. Simple Sample Output

```
Number of passengers: 2
Distance in km: 100
Passenger 1
Age: 30
Class (sleeper/ac3/ac2):
sleeper Cancelled? (y/n): n
Fare due: ■70.00
Passenger 2
Age: 65
Class (sleeper/ac3/ac2):
ac3 Cancelled? (y/n): n
Fare due: ■136.00
Total fare for active bookings: ■206.00
```

9. Conclusion

The calculator demonstrates class fares, concessions, reservation fees, and cancellation calculations. Production use must follow current official railway fare and refund rules.

6. Railway Fare and Reservation Calculator

10. Output Screenshot

Screenshot of the program run in a terminal using the sample input from Section 8.



Figure 6: Console output of Program 6 (Railway Fare and Reservation Calculator).

7. Retail Billing with Multiple Discount Rules

1. Problem Statement

Create itemized retail invoices for multiple products and quantities, applying category-specific taxes, threshold discounts, and a maximum concession limit.

2. Objectives

1. Calculate line totals and category taxes.
2. Apply quantity-independent invoice discounts based on subtotal thresholds.
3. Cap total discounts at a maximum concession percentage.
4. Generate an itemized invoice.

3. Assumptions

• Electronics tax: 18%; groceries tax: 5%; clothing tax: 12%.

• Invoice discount: 5% when subtotal reaches ■5,000; 10% when subtotal reaches ■10,000.

• Discount is capped at 8% of subtotal.

• For simplicity, invoice discount is applied proportionally before tax.

4. Algorithm

1. Input product name, category, unit price, and quantity.
2. Validate price and quantity.
3. Calculate line subtotal and invoice subtotal.
4. Determine threshold discount and apply cap.
5. Allocate discount proportionally to each line.
6. Calculate category tax on discounted line values.
7. Print itemized totals and invoice total.

5. Pictorial Flowchart

START ↓ Input products and categories ↓ Validate prices and quantities ↓ Calculate subtotal ↓ Calculate threshold discount and apply cap ↓ Apply category taxes ↓ Print itemized invoice ↓ END

6. Python Source Code

```
TAX_RATES = {"electronics": 0.18, "groceries": 0.05, "clothing": 0.12}

def main():
    n = int(input("Number of products:
    ")) if n <= 0:
        print("Enter at least one product.")
        return
    items = []
    subtotal = 0.0
    for i in range(n):
        print(f"Product {i+1}")
        name = input("Name: ")
        category = input("Category (electronics/groceries/clothing):
        ").lower() price = float(input("Unit price: ■"))
        qty = int(input("Quantity: "))
        if category not in TAX_RATES or price < 0 or qty <=
            0: print("Invalid category, price, or quantity.")
            return
        line = price *
        qty subtotal +=
        line
        items.append({"name": name, "category":
                      category, "line": line})
    discount_rate = 0.10 if subtotal >= 10000 else
        ( 0.05 if subtotal >= 5000 else 0.0)
    discount = min(subtotal * discount_rate, subtotal * 0.08)
    factor = (subtotal - discount) / subtotal if subtotal
    else 1 tax_total = 0.0
    print("\nITEMIZED INVOICE")
    print("Item | Category | Subtotal | Discounted |
    Tax") for item in items:
```

```
        discounted = item["line"] * factor
        tax = discounted * TAX_RATES[item["category"]]
        tax_total += tax
        print(f'{item["name"]} | {item["category"]} | '
              f'■{item["line"]:.2f} | ■{discounted:.2f} |
    ■{tax:.2f}') print(f"Subtotal: ■{subtotal:.2f}")
    print(f"Discount: ■{discount:.2f}")
    print(f"Tax: ■{tax_total:.2f}")
    print(f"Grand total: ■{subtotal-discount+tax_total:.2f}")

if    name    == "   main
      ": main()
```

7. Test Cases

Case	Input / condition	Expected result	Purpose
TC1	Groceries ■100 × 2	Subtotal ■200	5% tax = ■10
TC2	Subtotal ■5,000	5% nominal discount	Capped at 8%
TC3	Subtotal ■10,000	10% nominal discount	Actual cap 8%
TC4	Unknown category	—	Reject input
TC5	Quantity 0	—	Reject input
TC6	Mixed categories	Different tax rates	Calculate per line

8. Simple Sample Output

```
Number of products: 2
Product 1: Notebook, groceries, unit price ■100, quantity 2
Product 2: Shirt, clothing, unit price ■1000, quantity 1

ITEMIZED INVOICE
Notebook | groceries | ■200.00 | ■200.00 |
■10.00 Shirt | clothing | ■1000.00 | ■1000.00
| ■120.00 Subtotal: ■1200.00
Discount: ■0.00
Tax: ■130.00
Grand total: ■1330.00
```

9. Conclusion

The program combines category taxes and capped threshold discounts in an itemized invoice. Businesses should adapt tax categories, discount sequencing, and rounding to local rules.

7. Retail Billing with Multiple Discount Rules

10. Output Screenshot

Screenshot of the program run in a terminal using the sample input from Section 8.

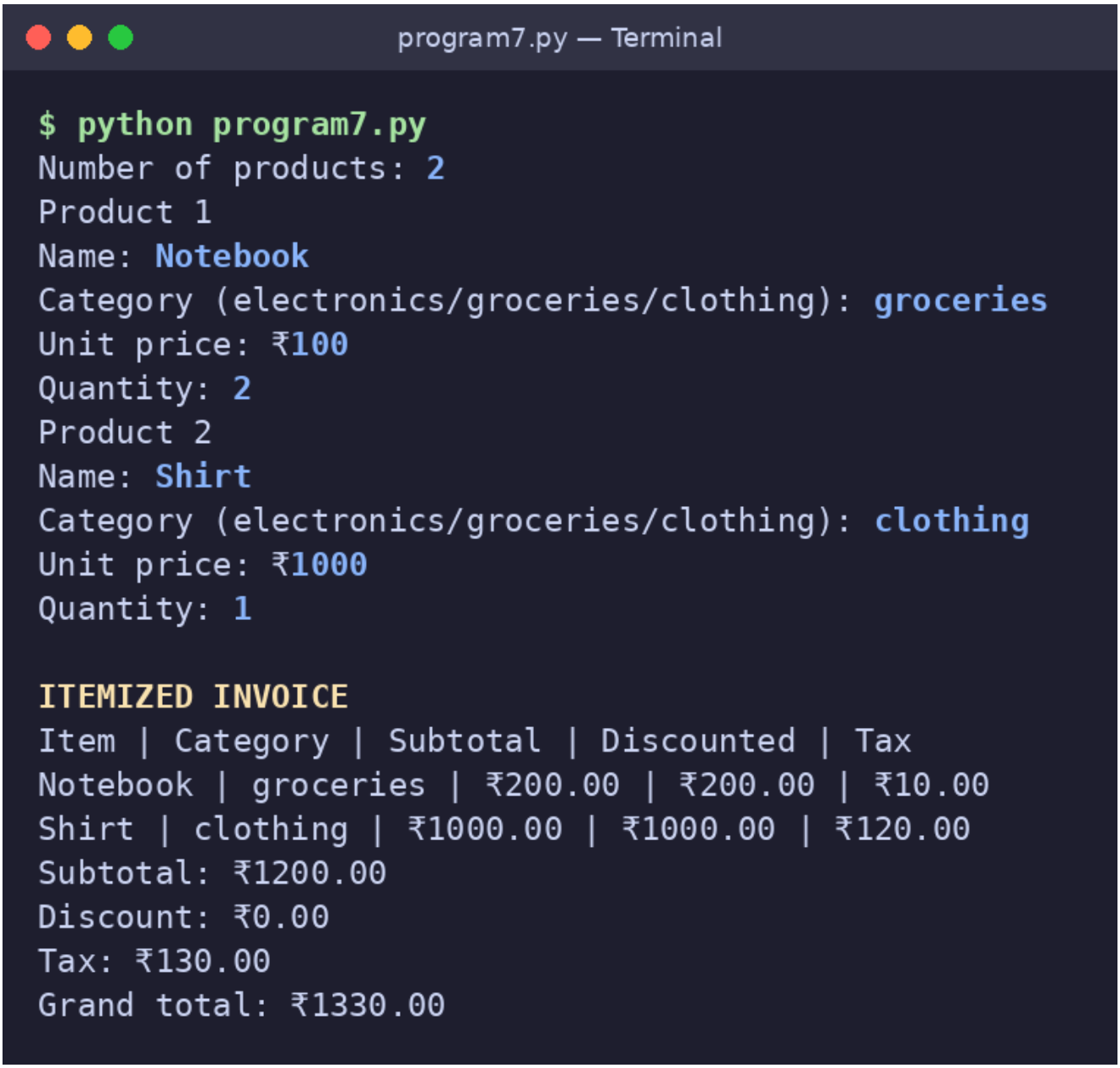


Figure 7: Console output of Program 7 (Retail Billing with Multiple Discount Rules).

8. Manufacturing Cost and Profitability Analysis

1. Problem Statement

Calculate material costs, labour, wastage, machine costs, and selling price to determine whether a production batch meets a specified profit target.

2. Objectives

1. Compute material cost and wastage.
2. Add labour and machine costs.
3. Calculate total production cost and unit cost.
4. Calculate revenue, profit margin, and target status.

3. Assumptions

• Wastage is a percentage of base material cost.

• Total cost = material + wastage + labour + machine costs.

• Revenue = units produced × selling price per unit.

• Profit target is specified as a percentage of total cost.

• All inputs must be non-negative; units produced must be positive.

4. Algorithm

1. Input units, material cost, wastage rate, labour, machine cost, selling price, and target profit rate.
2. Validate values.
3. Calculate wastage and total production cost.
4. Calculate revenue and profit.
5. Calculate profit percentage over cost.
6. Compare actual profit percentage to target and report result.

5. Pictorial Flowchart

START ↓ Input production and cost details ↓ Validate inputs ↓ Calculate material wastage and total cost ↓ Calculate revenue and profit ↓ Compare profit percentage with target ↓ Display profitability report ↓ END

6. Python Source Code

```
def main():
    units = int(input("Units produced: "))
    material = float(input("Base material cost: ■"))
    wastage_rate = float(input("Wastage percentage:
    ")) labour = float(input("Total labour cost: ■"))
    machine = float(input("Machine cost: ■"))
    selling = float(input("Selling price per unit: ■"))
    target = float(input("Target profit percentage on cost: "))
    if (units <= 0 or min(material, wastage_rate, labour, machine,
                          selling, target) <
        0): print("Invalid input.")
        return
    wastage = material * wastage_rate / 100
    total_cost = material + wastage + labour + machine
    revenue = units * selling
    profit = revenue - total_cost
    margin = profit / total_cost * 100 if total_cost else 0
    print("\nPRODUCTION REPORT")
    print(f"Material cost: ■{material:.2f}")
    print(f"Wastage cost: ■{wastage:.2f}")
    print(f"Labour cost: ■{labour:.2f}")
    print(f"Machine cost: ■{machine:.2f}")
    print(f"Total production cost:
    ■{total_cost:.2f}") print(f"Cost per unit:
    ■{total_cost/units:.2f}") print(f"Revenue:
    ■{revenue:.2f}") print(f"Profit/Loss:
    ■{profit:.2f}")
    print(f"Profit on cost: {margin:.2f}%")
    print("Target met" if margin >= target else "Target not

met") if    name    == "   main   ":
```

```
main()
```

7. Test Cases

Case	Input / condition	Expected result	Purpose
TC1	100 units; total cost ■8,000; revenue ■ 10,000	Profit ■2,000	25% profit on cost
TC2	Revenue equals cost	Profit ■0	Target 0% met
TC3	Revenue below cost	Negative profit	Target not met
TC4	Units = 0	—	Reject input
TC5	Wastage 10% on ■5,000	Wastage ■500	Included in cost
TC6	Target 30%, actual 25%	—	Target not met

8. Simple Sample Output

```
Units produced: 100
Base material cost:
■5000 Wastage
percentage: 10 Total
labour cost: ■2000
Machine cost: ■500
Selling price per unit: ■100
Target profit percentage on cost: 20

PRODUCTION REPORT
Material cost: ■5000.00
Wastage cost: ■500.00
Labour cost: ■2000.00
Machine cost: ■500.00
Total production cost:
■8000.00 Cost per unit: ■80.00
Revenue: ■10000.00
Profit/Loss: ■2000.00
Profit on cost: 25.00%
Target met
```

9. Conclusion

The analysis helps manufacturers understand cost structure, wastage, and profitability. Accurate production records and overhead allocation are essential for business decisions.

8. Manufacturing Cost and Profitability Analysis

10. Output Screenshot

Screenshot of the program run in a terminal using the sample input from Section 8.

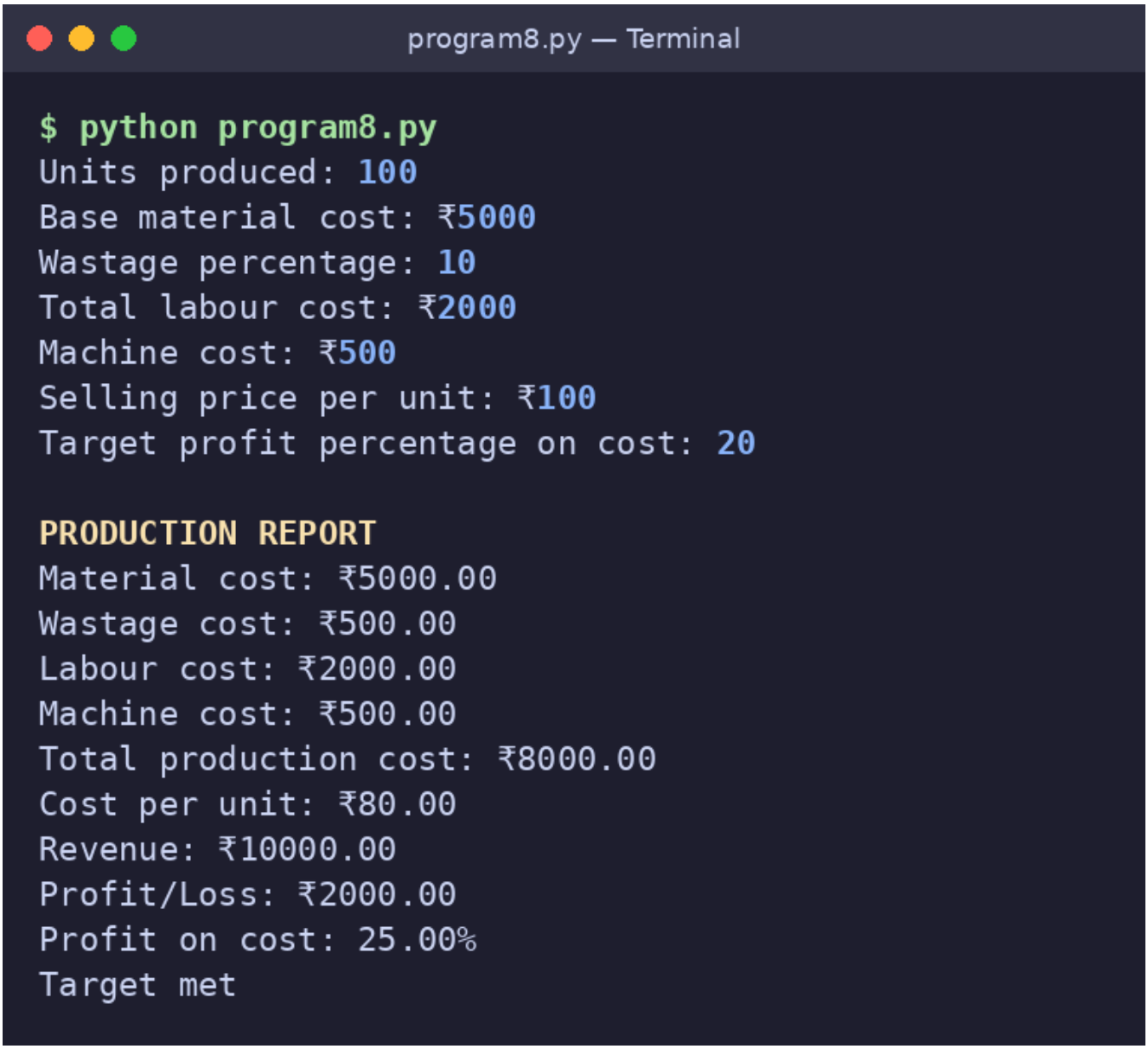


Figure 8: Console output of Program 8 (Manufacturing Cost and Profitability Analysis).

9. Multi-Currency Travel Budget Analysis

1. Problem Statement

Convert travel expenses in multiple currencies using supplied exchange rates, include conversion fees, compare spending against a budget, and identify the largest expense category.

2. Objectives

1. Convert foreign-currency expenses into INR.
2. Apply a currency conversion fee.
3. Compare actual spending with budget.
4. Find the largest spending category.

3. Assumptions

• The user supplies the exchange rate as INR per one foreign currency unit.

• Conversion fee is a percentage of converted INR.

• Expenses are grouped into categories.

• Exchange rates in the sample are illustrative and should be updated before travel.

4. Algorithm

1. Input budget and conversion fee percentage.
2. For each expense, input category, amount, currency, and INR exchange rate.
3. Convert expense and add fee.
4. Accumulate category totals and overall spending.
5. Compare total with budget and identify the largest category.

5. Pictorial Flowchart

START ↓ Input budget and fee rate ↓ Input each expense and exchange rate ↓ Convert amount to INR and add fee ↓

Accumulate category totals ↓ Compare with budget and find largest category ↓ Print summary ↓ END

6. Python Source Code

```
def main():
    budget = float(input("Travel budget in INR: ■"))
    fee_rate = float(input("Conversion fee percentage:
    ")) n = int(input("Number of expenses: "))
    if budget < 0 or fee_rate < 0 or n <= 0:
        print("Invalid input.")
        return
    totals = {}
    overall = 0.0
    for i in range(n):
        print(f"Expense {i+1}")
        category = input("Category: ").strip()
        amount = float(input("Amount in foreign currency: "))
        currency = input("Currency code: ").upper()
        rate = float(input(f"INR per 1 {currency}:
        ")) if amount < 0 or rate <= 0 or not
        category:
            print("Invalid expense.")
            return
        converted = amount * rate
        fee = converted * fee_rate / 100
        final = converted + fee
        totals[category] = totals.get(category, 0) +
        final overall += final
        print(f"Converted: ■{converted:.2f}; fee: ■{fee:.2f}; "
              f"total: ■{final:.2f}")
    print("\nTRAVEL BUDGET SUMMARY")
    for category, value in sorted(totals.items(),
                                  key=lambda x: x[1], reverse=True):
        print(f"{category}: ■{value:.2f}")
    print(f"Total spending: ■{overall:.2f}")
    print(f"Budget: ■{budget:.2f}")
    print(f"Remaining/over budget: ■{budget-overall:.2f}")
    print("Largest category:", max(totals, key=totals.get))

if    name    == "   main   ":
```

```
main()
```

7. Test Cases

Case	Input / condition	Expected result	Purpose
TC1	100 USD at ■83; fee 2%	▪ 8,466	Converted ■8,300 + ■166
TC2	Budget ■20,000; spend ■15,000	▪ 5,000 remaining	Within budget
TC3	Spend ■22,000; budget ■20,000	▪ 2,000 over	Over budget
TC4	Rate 0	—	Reject input
TC5	Two expenses in same category	Combine totals	Category aggregation
TC6	Unequal category totals	Largest category	Correct maximum

8. Simple Sample Output

```
Travel budget in INR: ■20000
Conversion fee percentage: 2
Number of expenses: 2

Expense 1: Hotel; amount 100; currency USD; INR rate
83 Converted: ■8300.00; fee: ■166.00; total:
■8466.00 Expense 2: Food; amount 50; currency EUR;
INR rate 90 Converted: ■4500.00; fee: ■90.00; total:
■4590.00

TRAVEL BUDGET SUMMARY
Hotel: ■8466.00
Food: ■4590.00
Total spending: ■13056.00
Budget: ■20000.00
Remaining/over budget:
■6944.00 Largest category:
Hotel
```

9. Conclusion

The tool standardizes multi-currency budget estimates and highlights the largest expense category. Users should use current exchange rates and check card-provider fees before travel.

9. Multi-Currency Travel Budget Analysis

10. Output Screenshot

Screenshot of the program run in a terminal using the sample input from Section 8.

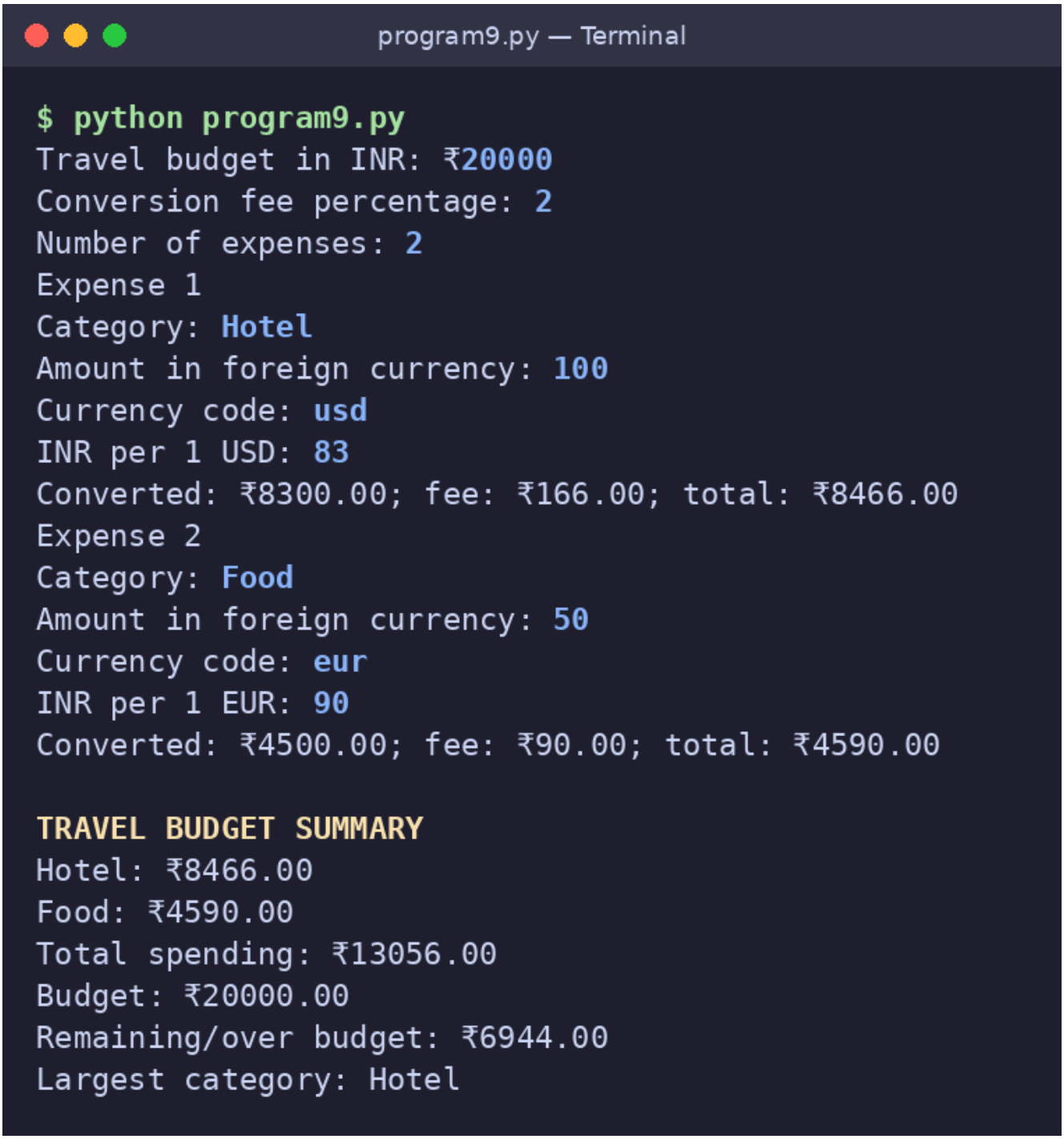


Figure 9: Console output of Program 9 (Multi-Currency Travel Budget Analysis).

10. Campus Event Financial Planning

1. Problem Statement

Estimate venue, equipment, catering, transport, and certificate costs; calculate participant contributions and sponsorship shortfalls; and determine the minimum registrations needed to break even.

2. Objectives

1. Calculate total fixed and variable event costs.
2. Calculate net cost after sponsorship.
3. Estimate contribution per participant.
4. Determine break-even registrations and sponsorship shortfall.

3. Assumptions

• Venue, equipment, and transport are fixed costs.

• Catering and certificates are variable per participant.

• Registration fee is fixed per participant.

• Break-even count is the ceiling of (fixed costs − sponsorship) divided by (registration fee − per-person variable cost), if contribution is positive.

4. Algorithm

1. Input fixed costs, per-person costs, sponsorship, and registration fee.
2. Validate values.
3. Calculate fixed cost and variable cost per participant.
4. Calculate break-even registration count.
5. For expected registrations, calculate total cost, revenue, and funding shortfall/surplus.
6. Display the event budget.

5. Pictorial Flowchart

START ↓ Input event costs and funding ↓ Validate values ↓ Calculate fixed and per-person costs ↓ Calculate contribution margin and break-even count ↓ Calculate expected event surplus/shortfall ↓ Print budget report ↓ END

6. Python Source Code

```
import math

def main():
    venue = float(input("Venue cost: ■"))
    equipment = float(input("Equipment cost:
    ■"))
    transport = float(input("Transport fixed cost: ■"))
    catering = float(input("Catering per participant: ■"))
    certificate = float(input("Certificate per participant:
    ■")) sponsorship = float(input("Sponsorship confirmed:
    ■"))
    fee = float(input("Registration fee per participant: ■"))
    expected = int(input("Expected registrations: "))
    values = [venue, equipment, transport, catering,
              certificate, sponsorship, fee]
    if min(values) < 0 or expected < 0:
        print("Values cannot be negative.")
        return
    fixed = venue + equipment + transport
    variable = catering + certificate
    contribution = fee - variable
    if contribution > 0:
        needed = max(0, math.ceil((fixed - sponsorship) /
    contribution)) else:
        needed = 0 if sponsorship >= fixed else None
    total_cost = fixed + variable * expected
    revenue = sponsorship + fee * expected
    balance = revenue - total_cost
    print("\nEVENT FINANCIAL PLAN")
    print(f"Fixed costs: ■{fixed:.2f}")
    print(f"Variable cost per participant: ■{variable:.2f}")
```

```
print(f"Contribution per registration:
■{contribution:.2f}") print("Break-even registrations:",
```

```
        needed if needed is not None else "Not possible at this fee")
    print(f"Total cost at {expected} registrations: ■{total_cost:.2f}")
    print(f"Funding including sponsorship: ■{revenue:.2f}")
    print(f"Surplus/shortfall: ■{balance:.2f}")
    print(f"Sponsorship shortfall against fixed costs:
        " f"■{max(0, fixed-sponsorship):.2f}")

if    name    == "   main
      ": main()
```

7. Test Cases

Case	Input / condition	Expected result	Purpose
TC1	Fixed ■10,000; variable ■100; fee ■200; sponsorship ■2,000	Break-even 80 registrations	Contribution ■100
TC2	Fee equals variable cost; sponsorship below fixed cost	No finite break-even	Contribution ■0
TC3	Sponsorship covers fixed cost	Break-even 0 fixed-cost gap	Still cover variable costs
TC4	Negative venue cost	—	Reject input
TC5	Expected registrations 100	Calculate total cost/revenue	Show surplus or shortfall
TC6	Sponsorship ■0	Shortfall equals fixed costs	Before registration income

8. Simple Sample Output

```
Venue cost: ■10000
Equipment cost: ■2000
Transport fixed cost:
■3000
Catering per participant: ■150
Certificate per participant: ■20
Sponsorship confirmed: ■5000
Registration fee per participant: ■300
Expected registrations: 50

EVENT FINANCIAL PLAN
Fixed costs: ■15000.00
Variable   cost   per   participant:
■170.00       Contribution       per
registration:   ■130.00   Break-even
registrations: 77
Total cost at 50 registrations: ■23500.00
Funding including sponsorship: ■20000.00
Surplus/shortfall: ■-3500.00
Sponsorship shortfall against fixed costs: ■10000.00
```

9. Conclusion

The event planner estimates break-even attendance and funding gaps before commitments are made. Estimates should be updated with supplier quotes and realistic registration forecasts.

10. Campus Event Financial Planning

10. Output Screenshot

Screenshot of the program run in a terminal using the sample input from Section 8.

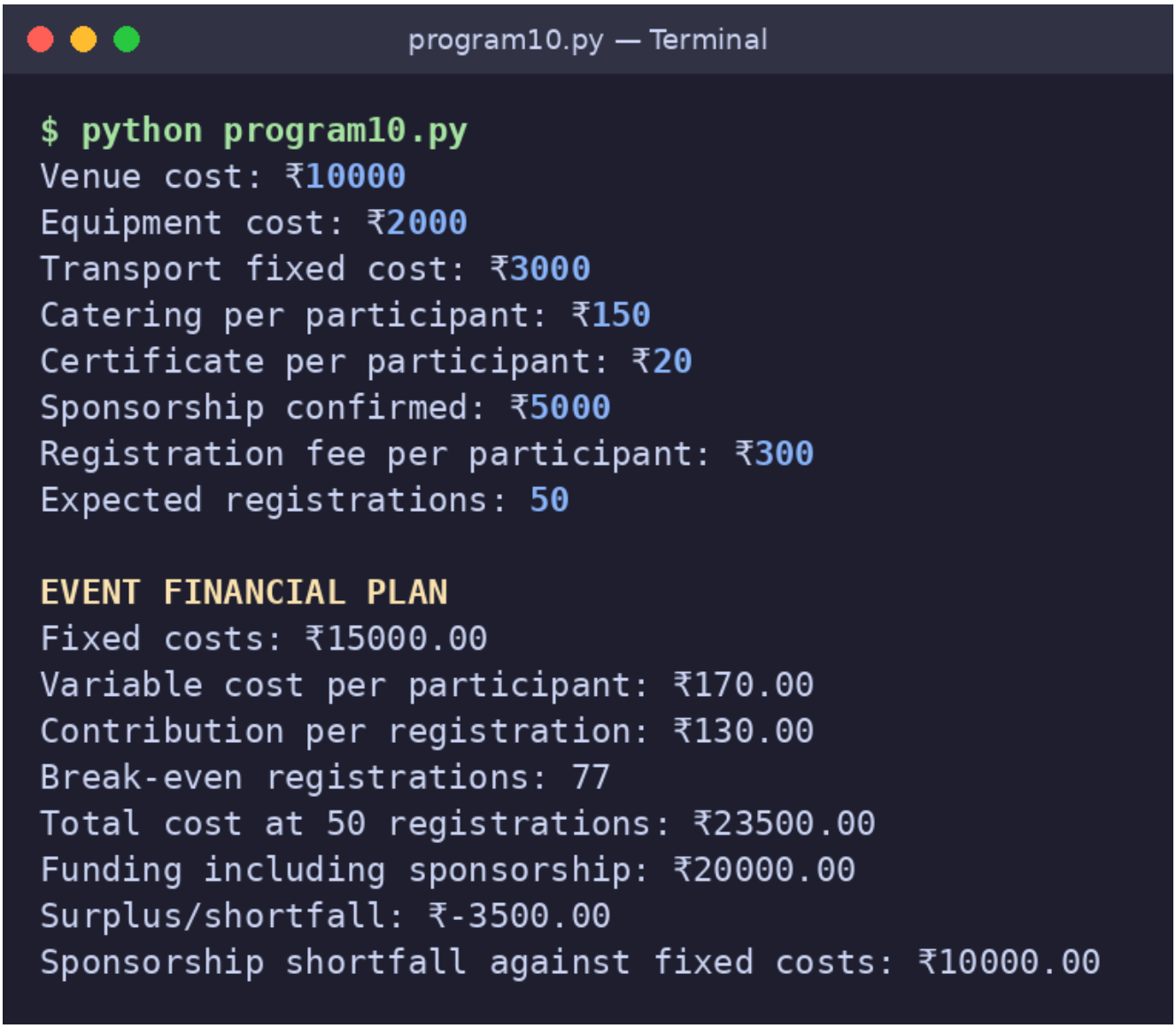


Figure 10: Console output of Program 10 (Campus Event Financial Planning).